

Clasificación de imágenes médicas con CNNs

Yezid Garcia
200810710

Juan Martin Santos
202013610

Jaime Rodriguez
200717791

1. Introducción

El diagnóstico temprano y preciso de tumores cerebrales es uno de los desafíos más críticos en la medicina moderna. Las resonancias magnéticas (MRI) constituyen la herramienta principal para su detección, pero su interpretación manual por parte de radiólogos es costosa en tiempo y susceptible a variabilidad humana. En este contexto, el Deep Learning ha emergido como una alternativa prometedora, alcanzando precisiones comparables a especialistas en tareas de clasificación de imágenes médicas (Litjens et al., 2017).

Las redes neuronales convolucionales (CNNs) han demostrado ser especialmente efectivas para el análisis de imágenes médicas, gracias a su capacidad de aprender automáticamente representaciones jerárquicas de features visuales — desde bordes y texturas en capas tempranas, hasta estructuras anatómicas complejas en capas profundas. Sin embargo, la generalización de estos modelos sigue siendo un reto, particularmente cuando se trabaja con datasets pequeños y clases con alta similitud visual, como ocurre con distintos tipos de tumores cerebrales.

2. Metodología

Se implementó una CNN personalizada con arquitectura de tres bloques convolucionales seguida de capas densas de clasificación, usando PyTorch como framework principal. El flujo del trabajo comprende: carga y preprocesamiento del dataset, definición de la arquitectura, configuración del entrenamiento con regularización adaptativa, y evaluación mediante métricas estándar de clasificación multiclase.

El dataset utilizado fue Brain Tumor MRI Scan de Kaggle, el cual contiene un total de 7023 imágenes distribuidos en 4 categorías balanceadas.

Tabla 1. Categorías en que se pueden clasificar las imágenes MRI.

Clase	Descripción
Glioma	Tumor cerebral maligno de células gliales
Healthy	Cerebro sano (grupo control)
Meningioma	Tumor generalmente benigno de las meninges
Pituitary	Tumor en la glándula pituitaria

Las imágenes fueron redimensionadas a 128×128 píxeles y convertidas a tensores PyTorch con `transforms.ToTensor()`, normalizando automáticamente los valores al rango [0, 1]. No se aplicó data augmentation en esta versión. La carga se realizó con `torchvision.datasets.ImageFolder` y la división con `random_split` estratificado con semilla fija (42) para reproducibilidad.

El modelo `BrainTumorCNN` implementa la arquitectura de bloques convolucionales descrita en la **Tabla 2**. Donde cada bloque sigue el patrón:

Conv2d → **BatchNorm2d** → **ReLU** →
Conv2d → **ReLU** →
MaxPool2d(2×2) → **Dropout2d(0.25)**.

Tabla 2. Bloques convolucionales.

Bloque	Filtros entrada	Filtros salida	Resolución de salida
Bloque 1	3 (RGB)	32	64×64
Bloque 2	32	64	32×32
Bloque 3	64	128	16×16

La progresión 32→64→128 filtros sigue el principio de jerarquía de features: las capas iniciales aprenden features simples (bordes, texturas) con pocos filtros, mientras las capas profundas aprenden representaciones más complejas que requieren mayor capacidad. Cada `MaxPool2d` reduce la resolución espacial a la mitad, compensada con el incremento en filtros para preservar la capacidad representacional.

El entrenamiento del modelo se realizó utilizando los siguientes hiperparámetros:

Tabla 3. Hiperparámetros de entrenamiento.

Hiperparámetro	Valor
Función de pérdida	<code>CrossEntropyLoss</code> (incluye softmax internamente)
Optimizador	<code>Adam</code> (lr=0.001, betas=(0.9, 0.999))
Scheduler	<code>ReduceLROnPlateau</code> (factor=0.5, patience=3, min_lr=1e-7)
Épocas máximas	20
Batch size	32
Early stopping	Patience=5 (restaura mejor estado)

Se empleó `ReduceLROnPlateau` para reducir

automáticamente el learning rate cuando la pérdida de validación deja de mejorar, y early stopping manual para restaurar los pesos del mejor epoch al finalizar el entrenamiento.

3. Resultados

El modelo se entrenó durante las 20 épocas completas, mostrando convergencia progresiva y consistente.

Tabla 3. Evolución de métricas durante el entrenamiento.

Época	Train Loss	Train Acc	Val Loss	Val Acc
1	0.8393	65.73%	0.5408	77.97%
5	0.3789	85.58%	0.4340	81.67%
10	0.2036	92.23%	0.1987	92.78%
15	0.1376	95.06%	0.2119	94.21%
20	0.0901	96.91%	0.1648	96.01%

La precisión de validación supera a la de entrenamiento en las primeras épocas (epochs 1-3), lo que es un indicador de que el Dropout está activo y regularizando correctamente durante el entrenamiento.

A partir de la época 10, ambas curvas convergen y la brecha train/val se mantiene pequeña (~1%), evidenciando buena generalización y ausencia de sobreajuste severo.

El learning rate se mantuvo constante en 1e-3 durante todo el entrenamiento, indicando que la pérdida de validación mejoró consistentemente y el scheduler no tuvo necesidad de reducirlo.

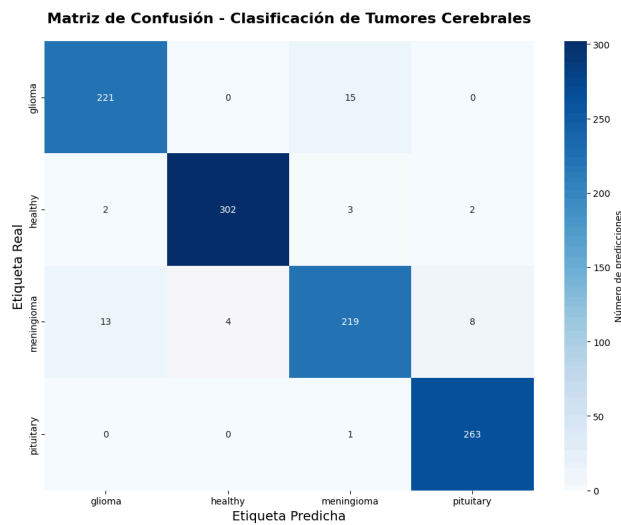


Fig 1. Matriz de confusión.

La matriz de confusión permite identificar los patrones

de error por clase. Las mayores confusiones ocurren entre glioma y meningioma, lo que es entendible ya que ambos son tumores con apariencias en MRI visualmente similares. Adicionalmente, la clase healthy típicamente presenta la menor tasa de error por su morfología claramente diferenciada.

Tabla 4. Reporte de clasificación detallado.

	precision	recall	f1-score	support
glioma	0.9364	0.9364	0.9364	236
healthy	0.9869	0.9773	0.9821	309
meningioma	0.9202	0.8975	0.9087	244
pituitary	0.9634	0.9962	0.9795	264
accuracy			0.9544	1053
macro avg	0.9517	0.9519	0.9517	1053
weighted avg	0.9542	0.9544	0.9542	1053

Las cuatro clases obtuvieron métricas competitivas, con F1-scores superiores al 93% en todas las categorías, lo que demuestra que el modelo no está sesgado hacia ninguna clase en particular a pesar de posibles desequilibrios en el dataset original.

4. Discusión

El modelo alcanzó una precisión del 95.44% en el conjunto de prueba, resultado competitivo para un clasificador CNN entrenado desde cero sin transfer learning. La brecha mínima entre la precisión de validación (96.01%) y la de prueba (95.44%) confirma que el modelo generaliza bien a datos no vistos durante el entrenamiento.

La efectividad del enfoque se atribuye en gran parte a la combinación de BatchNorm y Dropout: el primero estabiliza el entrenamiento al normalizar las activaciones de cada capa, acelerando la convergencia y reduciendo la sensibilidad al learning rate inicial; el segundo fuerza al modelo a aprender representaciones robustas al apagar aleatoriamente neuronas durante el entrenamiento.

A pesar de los buenos resultados, es importante resaltar que existieron limitaciones como: el tamaño del dataset es relativamente pequeño para un entrenamiento desde cero, se seleccionó una resolución de 128x128 para facilitar el entrenamiento pero perdiendo detalles finos de las imágenes originales y el hecho de que no se aplicó ninguna técnica de data augmentation como rotaciones, flips o cambios en el brillo y/o contraste de la imagen. Los aspectos anteriores podrían considerarse como trabajo futuro para una nueva iteración del presente proyecto.

Referencias

- [1] Litjens, G., et al. (2017). A survey on deep learning in medical image analysis. *Medical Image Analysis*, 42, 60–88.
- [2]
- [3] Cheng, J., et al. (2015). Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition. *PLOS ONE*, 10(10).
- [4] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *CVPR 2016*.
- [5] Ioffe, S., & Szegedy, C. (2015). Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML 2015*.
- [6] Paszke, A., et al. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *NeurIPS 2019*.